

Enabling Federated Learning for Object Detection in Connected Autonomous Driving Using YOLO with the Flower Framework

Komala Subramanyam Cherukuri, Kewei Sha, and Junhua Ding
University of North Texas, USA

Abstract—Connected autonomous vehicles (CAVs) rely on object detection models to ensure safe and efficient navigation. Traditional centralized training approaches pose challenges related to data privacy, scalability, and communication overhead. In this study, we integrate Federated Learning (FL) with YOLO models, including YOLOv5, YOLOv8, and YOLOv11, for object detection in CAVs, utilizing the Flower framework to enable decentralized training while preserving data privacy. We design a virtual client setup that replicates a realistic scenario and apply FedAvg and FedProx aggregation strategies on the KITTI and BDD100K datasets. Our experimental results demonstrate that FL-based training outperforms traditional centralized learning, with YOLOv8 achieving a mean average precision (mAP) of 87.9% in KITTI and 61.5% in BDD100K, outperforming the baseline. Our study highlights the feasibility and effectiveness of deploying FL-based object detection models in CAVs, by conducting a comprehensive evaluation of using the Flower federated learning framework and addressing privacy concerns through decentralized training.

I. INTRODUCTION

Connected Autonomous Vehicles (CAVs) are transforming transportation by enabling safer and more efficient mobility. A core component is object detection, which allows CAVs to perceive their surroundings in real time by identifying and localizing relevant objects to support decisions like trajectory planning and motion control [1]. To achieve this, CAVs use deep learning (DL) to process high-resolution data from cameras and LiDAR, enabling precise object recognition and better navigation [2]. The vast amounts of data generated by autonomous vehicles (AVs), estimated at approximately one gigabyte per second, present significant challenges in data processing and privacy preservation [3]. Traditional centralized machine learning approaches face challenges when dealing with diverse and complex data from multiple CAVs, often requiring transferring large volumes of data to a central server, and raising concerns about privacy, scalability, and high communication costs [4]. FL addresses these limitations by allowing local model updates on distributed devices, thereby preserving data privacy and significantly reducing bandwidth usage [5]. The architecture of FL, as illustrated in Figure 1, inherently reduces network strain by transmitting only essential model data between the server and clients, without exposing local datasets [6]. As a decentralized learning paradigm, FL improves performance in non-independent and identically distributed (non-IID) and unbalanced data by leveraging efficient client selection and aggregation strategies, ensuring robust learning despite varying data distributions across clients [7].

Existing research in FL for autonomous driving has pri-

marily focused on optimizing aggregation strategies, performance, and privacy, but significant challenges remain in making FL-based object detection feasible for real-world CAVs. These challenges extend beyond algorithmic optimizations to include support for large-scale system deployment and rigorous performance evaluation that explores trade-offs between learning performance and system resource consumption. Given the prohibitive cost of testing and evaluating FL models directly in real AVs, establishing a controlled test environment is crucial before the real-world implementation. Several frameworks support FL implementation, including TensorFlow Federated (TFF) and Flower. TFF is primarily designed for use with TensorFlow, which limits its flexibility in handling resource constraints and communication challenges across varied environments. Flower overcomes these limitations with a flexible, framework-agnostic design that supports heterogeneous clients, scalable training, and simulation of deployment constraints, bridging the gap between research and real-world implementation [8]. Despite its advantages, the existing Flower framework lacks embedded mechanisms to collect essential resource utilization data, making it challenging to analyze the computational and communication overheads associated with FL in CAVs. Additionally, integrating Flower with existing object detection models is nontrivial, requiring extensive customization to ensure compatibility with real-world CAV workloads.

We address the above gaps by proposing and implementing a FL-based object detection framework, integrating popular object detection models such as YOLOv5, YOLOv8 and YOLOv11, the powerful Flower FL framework with FedAvg and FedProx as aggregation strategies, and resource monitoring tools. We also examine the challenges and insights observed during integration and in autonomous driving setup. The **key contributions** of this paper are three-fold. First, we integrated YOLO models with the Flower framework, enabling decentralized object detection. Our setup helps simulate and analyze federated training before real-world deployment using AVs. Second, we designed a virtual client setup that mimics real-world autonomous driving scenarios, handling non-IID and imbalanced data. This makes federated training more practical and effective for edge deployment. Third, we conducted a comprehensive evaluation by testing three YOLO models with Flower on two datasets using FedAvg and FedProx. The results show that federated learning achieves the mAP of 87.9% on KITTI and 61.5% on BDD100K while maintaining privacy and reducing communication overhead. The rest of this paper is structured as

follows. Section II reviews existing work using FL in CAVs. Section III details a new FL design that integrates YOLO with the Flower framework. Section IV describes the datasets and experiment setup, and presents experimental results. The conclusion is presented in Section V.

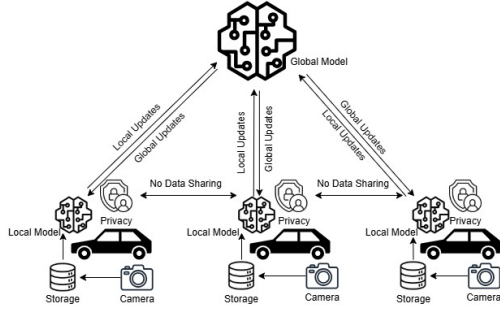


Fig. 1: Federated Learning for Object Detection in AV.

II. BACKGROUND AND MOTIVATION

FL has been explored for object detection in CAV to address challenges such as data privacy, model generalization, and decentralized training. Various studies have focused on optimizing aggregation strategies, improving privacy and security, and handling non-IID data [5].

Researchers explored FL in autonomous driving to enhance privacy and improve object detection, developing frameworks that make model training more efficient and adaptable. A TensorFlow-based FL framework using a client-server model with socket communication, FedAvg aggregation, and YOLOv3 for object detection in autonomous vehicles was implemented in [2]. Global road damage detection is addressed in [9] by applying FL with YOLOv5. FL has also been employed to enhance security and privacy in autonomous driving. An FL-based platform for object recognition in CAVs was developed in [10], reducing data exposure and ensuring continuous model adaptation. To enhance vehicular privacy, the authors in [11] introduced an FL system with edge computing, incorporating traceable identity-based encryption and zero-knowledge proofs while maintaining performance. VINO_EffiFedAV [12] presents an FL framework for real-time object detection in CAV, integrating selective client updates with FedAvg to improve robustness and efficiency in handling Non-IID data. A vision-centric FL platform, COALA [13], enables federated object detection and segmentation while addressing data heterogeneity and model customization. An end-to-end framework for decentralized model training with a two-stream CNN was presented in [14], reducing communication overhead and improving adaptability for autonomous vehicles. A federated semi-supervised learning framework combining labeled and unlabeled data to improve detection performance, especially in low-labeled scenarios, was proposed in [15].

Several FL aggregation strategies have been proposed to enhance autonomous driving models under decentralized and heterogeneous conditions. FedDSL, a federated self-ensembling approach, was proposed in [16] to handle data heterogeneity in autonomous driving, achieving

near-supervised performance with unlabeled client data. To improve adaptability in non-IID settings, [17] introduces FedProx+LA, integrating label-aware aggregation with a proximal term to enhance continuous online training in vehicular networks. Privacy-preserving monocular depth estimation for autonomous vehicles was proposed in [18] through FedSCDepth, a self-supervised FL approach ensuring computational efficiency with performance comparable to centralized training. Sparse federated training for object detection in the Internet of Vehicles was introduced in [19] with FedWeg, dynamically adjusting sparsity ratios and using inverse sparsity weighting to reduce communication overhead while maintaining accuracy. Real-time object detection in the Internet of Vehicles is addressed by [20] through FedPylot, an MPI-based FL framework optimizing YOLOv7 under heterogeneous conditions while ensuring secure server-client communication.

In summary, existing work in FL for autonomous driving focusing on optimization and communication efficiency, flexible aggregation, and edge deployment capabilities. Our study integrates the Flower framework with YOLO for federated object detection in autonomous driving, with KITTI and BDD100K datasets. Flower's simulation environment enables controlled testing of federated YOLO training, optimizing aggregation strategies and assessing computational demands before deployment. This virtual setup enables testing and optimization for edge deployment. We evaluate Flower's effectiveness and highlight challenges, offering insights into its feasibility for object detection in autonomous systems.

III. THE INTEGRATED FRAMEWORK

To extend the existing work, our study focuses on the integration of YOLO with the flower framework, to evaluate its practical applicability for federated object detection in autonomous driving environments.

A. Federated Learning with the Flower Framework

The Flower framework offers a flexible architecture for federated learning (FL), making it well-suited for integrating YOLO models in decentralized object detection tasks. As shown in Figure 2, Flower supports virtualized FL environments that enable dynamic client instantiation and efficient model aggregation. This architecture allows training YOLO models across distributed, simulated clients, closely aligning with real-world autonomous driving scenarios.

The diagram illustrates the Flower framework facilitates training by coordinating a global model across multiple virtual clients. Each client holds its own local dataset and trains a local version of the model without sharing its data. After training, clients send only the model updates to the FL server, which then aggregates these updates using strategies like FedAvg or FedProx to improve the global model. This cycle of training and aggregation continues over multiple rounds. To ensure efficient execution, the Ray backend is used to run all clients in parallel. Once the global model is updated, it is evaluated using a shared test set to measure overall performance. This approach is especially effective

for training YOLO models in CAVs, where data privacy, distribution, and realism are key concerns.

A key challenge during integration was ensuring that all clients used an identical YOLO model architecture. Federated learning assumes uniformity in model structure across clients to aggregate updates correctly. Any variation in architecture such as differences in layer count or parameter shapes can lead to tensor shape mismatches during aggregation, causing Flower to raise size mismatch errors and disrupting the training process. To address this, a unified model was distributed to all clients. Each client imported this standardized model to initialize their local training, ensuring consistent parameter shapes and counts across the network. This strategy eliminated incompatibility issues and enabled smooth global model updates. Another limitation was Flower's lack of built-in tools for resource monitoring. This made it difficult to evaluate performance and manage computational load during training. To mitigate this, we integrate Flower with external tools such as psutil, pynvml for system monitoring and nvidia-smi/nvtop for real-time GPU usage tracking, allowing for assessment of resource utilization.

To enhance training robustness, model checkpoints (.pt files) were saved after each round to preserve intermediate states and enable recovery in case of interruptions. However, this increased storage requirements, as YOLO model weights are large and can exceed hundred MBs. The absence of built-in model compression in Flower led to increased communication and storage costs when transmitting model updates.

Our design introduced dynamic client instantiation, allowing for a more realistic and flexible training setup compared to static FL environments. A non-IID data partitioning strategy was to reflect real-world driving variability, enabling YOLO to train on diverse and imbalanced datasets. To reduce bandwidth, only serialized model updates were transmitted, maintaining YOLO's object detection performance while optimizing communication efficiency in the federated setting.

B. Components of the Proposed Framework

1) *YOLO*: YOLO [21] is a state-of-the-art object detection model that performs bounding box regression and classification within a single neural network. We utilize three representing YOLO models. YOLOv5 is the first version of YOLO implemented on PyTorch due to its popularity in federated learning studies. YOLOv8 is the first anchor free based model in YOLO versions, and YOLOv11 is the latest version used for object detection in autonomous driving.

2) *Aggregation Strategies*: FedAvg handles distributed optimization by coordinating decentralized clients to minimize a global objective function. FedProx extends this approach by introducing a proximal term to the loss function, addressing heterogeneity in data and system characteristics across clients. This modification improves training stability and convergence, particularly in non-IID (non-identically distributed) data settings. These two algorithms were selected because FedAvg is a widely adopted baseline in FL, while FedProx is better suited for the heterogeneous data chal-

lenges common in autonomous driving scenarios. Together, they enable a comparison between a general-purpose method and one specifically designed for non-IID environments.

3) *Client Setup*: Each virtual client simulates an autonomous vehicle, processing non-IID local data, where each client receives distinct image subsets. Clients are dynamically instantiated at runtime using Ray, with runtime referring to the execution phase when clients are created and assigned data. To reflect real-world autonomous driving systems where edge devices like onboard computers, roadside units, and embedded controllers operate under strict hardware constraints such as limited memory, CPU, and energy, dynamic client selection is used. Dynamic selection refers to the adaptive process of deciding which clients participate in each training round based on the system's real-time resource availability. Rather than forcing all clients to train regardless of capacity, it activates only those with sufficient resources. This prevents system overload, avoids unnecessary delays, and reflects real-world conditions where clients may intermittently drop out or become unavailable. This selection is achieved by configuring parameters such as fraction fit in the Flower framework to control client participation, and using client resources to define required CPU/GPU per client. The system then schedules clients based on available hardware at runtime. This setup also supports fault tolerance, allowing training to continue even when some clients fail or disconnect. By combining adaptive selection and resilience, this approach enhances scalability, ensures efficient resource utilization, and delivers realistic, robust federated learning simulations well-suited for autonomous driving deployment.

4) *Resource Monitoring & Management*: Efficient resource usage is critical in FL, edge devices in CAV, where clients often vary in hardware capacity. To manage this heterogeneity, our design combines Ray's task scheduling with real-time resource monitoring using psutil. Ray assigns training tasks based on predefined resources, ensuring that only clients with these static requirements are considered. Static declarations alone are insufficient to guarantee runtime reliability, clients may still have limited available memory due to background processes or system fluctuations. To overcome this, we incorporate psutil to monitor the actual system state of each client at runtime. Before each training round, the system performs real-time checks on each client's memory and CPU/GPU availability to determine whether it is suitable for participation. Only clients that meet these real-time thresholds are selected to participate. This dynamic filtering layer ensures that selected clients are not just eligible by configuration but also practically capable of executing the training task without failure or delay. By integrating psutil, we prevent runtime crashes, reduce waiting time, and improve the overall robustness and efficiency of the FL process, while closely reflecting practical edge deployment scenarios.

5) *Communication*: The server aggregates these updates using FedAvg and FedProx, refining the global model before redistributing it to clients. This iterative process enables continuous learning, promoting efficient knowledge transfer

and robust model generalization in CAV scenarios. After aggregation, the updated global model is sent back to all clients, enabling continuous learning across the network.

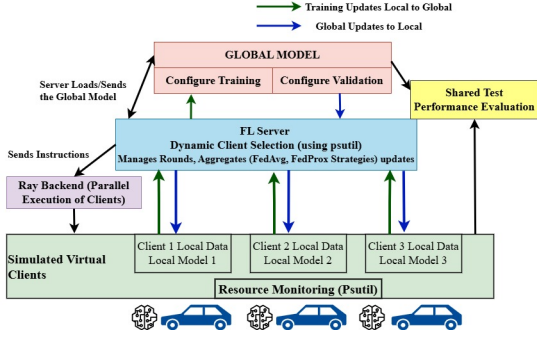


Fig. 2: YOLO in Federated Learning Setup with Flower

The integration of YOLO models within the Flower framework including model initialization, federated configuration, communication protocols, and dynamic resource management the subsequent section presents the experimental design, datasets, and evaluation metrics used to assess the performance of the federated object detection framework in autonomous driving scenarios.

IV. PERFORMANCE EVALUATION

A. DATA

In this study, we use the KITTI [22] and BDD100K [23] datasets, which reflect real-world autonomous driving challenges like class imbalance and non-IID data. The KITTI dataset includes 7,481 labeled training images and 7,518 test images, with multiple object classes (Table II). We use only the labeled training images. The BDD100K dataset provides 70,000 training, 10,000 validation, and 20,000 test images. Labels are available only for the training and validation sets, and object class details provided in Table III. The *train* class was excluded due to its limited instances(146 objects), which could lead to an imbalanced training process and affect model performance.

We only used images that had labels and separated them into training and testing sets. For the KITTI dataset, we used 5,985 labeled images for training and 1,496 for testing. In the BDD100K dataset, we used 70,000 labeled images for training and 10,000 for testing. These data sets were selected for their varied annotations, sizes, and applicability to tasks that involve object detection in autonomous driving systems, as shown in Table I. The BDD100K dataset was divided into

TABLE I: Comparison of Datasets for Object Detection

Dataset	Classes	Resolution	Weather	Day/Night	Location
KITTI	8	376p	No	No	Urban
BDD100K	10	720p	Yes	Yes	Urban

eight partitions to account for its significantly larger number of images, ensuring a diverse representation of environmental and object distributions. The KITTI dataset was split into four due to its relatively smaller size. We have partitioned the data as shown in Table II and III:

TABLE II: Count of each class KITTI

Class	C1	C2	C3	C4
Car	11508	5920	2925	2823
Van	1173	615	264	280
Truck	434	228	105	114
Pedestrian	1814	934	425	426
Person Sitting	117	29	7	17
Cyclist	636	321	170	162
Tram	210	92	28	92
Misc	395	207	91	89

TABLE III: Count of each class BDD100K

Class	C1	C2	C3	C4	C5	C6	C7	C8
Pedestrian	46279	22441	11381	5805	2690	1385	703	665
Rider	2325	1088	577	291	138	48	20	30
Car	356110	178724	88719	45365	21885	11284	5721	5403
Truck	14909	7507	3705	1935	976	476	228	235
Bus	5780	2968	1441	727	347	214	105	90
Motor	1449	736	403	225	118	32	17	22
Bike	3820	1698	926	446	164	76	35	45
Traffic Light	92792	46558	23342	11796	5659	3040	1495	1435
Traffic Sign	120510	59410	30131	14896	7307	3812	1787	1833

To introduce non-IID characteristics, the KITTI dataset was split by heavily concentrating the 'Car' class in Client 1 and distributing classes unevenly and Client 4 with less objects. In BDD100K, Client 1 contains the majority of samples for all classes, while Clients 7 and 8 hold minimal instances, simulating extreme data imbalance and environmental diversity. The dataset is partitioned to reflect real-world autonomous driving scenarios, where data is naturally imbalanced across locations and conditions. This non-IID (non-independent and identically distributed) setup enables the model to train on varied client data, each representing different environments. Simulating different driving environments in federated learning enables the model to train on diverse data, providing a more realistic evaluation of federated learning strategies. The approach also addresses key challenges such as data imbalance and class sparsity, by training the model on varied class distributions across clients, enabling it to learn to detect both frequently and infrequently occurring objects more effectively.

B. Experiment Setup and Evaluation Metrics

Our task is conducted on Nvidia H100 NVL GPUs. This powerful hardware configuration accelerates the processing and analysis of data, enhancing the speed.

We conducted experiments on the KITTI and BDD100K datasets, resizing all images to 640×640 for consistency. Using YOLOv5x6u, YOLOv8x, and YOLOv11x, we trained models with a batch size of 32 and a learning rate of 0.01 for 10 epochs in a centralized setting. For federated learning, we employed the Flower framework with decentralized data, running 3 local epochs per round over 10 communication rounds. In our work, we implemented YOLOv5, YOLOv8 and YOLOv11 for object detection within a federated learning setup using the Flower framework. The implementation consisted of two primary stages: model preparation and federated learning configuration.

Prior studies					Deep Learning			FedAvg			FedProx		
DATA	Method	YOLO	mAP	Reference	v5	v8	v11	v5	v8	v11	v5	v8	v11
KITTI	VINO_EffFedAV	v8l	84.3%	[12]	78.4%	81.6%	77.6%	82.9%	87.5%	84.0%	83.4%	87.9%	84.1%
	FedPylot	v7-W6	71.8%	[20]									
BDD	FedDSL	v5	61.3%	[16]	59.3%	61.4%	60.2%	58.9%	61.5%	60.5%	59.6%	61.5%	60.7%
	COALA	v5	34.4%	[13]									

TABLE IV: Comparison of Prior Results and Our Federated Learning Results

Model Preparation: We initialized our experiments with pre-trained YOLOv5, YOLOv8 and YOLOv11 models. These models were fine-tuned on KITTI and BDD100K datasets to adapt to object detection tasks.

Federated Learning Configuration: The federated learning setup involved setting up both the server and clients using the Flower framework. Each client was assigned a dataset, as shown in Table II and III, and contributed to the decentralized training process. The server is responsible for handling the aggregation of model updates to build a global model. To ensure efficient communication, model parameters were extracted and transmitted in a serialized NumPy array format, reducing bandwidth usage and speeding up processing. Each client loaded the YOLO model and conducted training, following a structured federated setup to ensure consistency in training and evaluation across all clients.

To evaluate the performance of the models, metrics including precision, recall, and mean Average Precision (mAP) are used. The Intersection Over Union (IoU) measures the overlap between predicted and ground truth bounding boxes. A detection is a True Positive (TP) if IoU exceeds a set threshold; otherwise, it is a False Positive (FP). Undetected ground-truth objects are False Negatives (FN). Precision and recall further evaluate detection accuracy and completeness. Average Precision (AP) quantifies the accuracy of object detectors at different recall levels. The mean Average Precision (mAP) extends the concept of AP to multiple classes and is a standard metric for datasets with multiple object categories. It is the average of the AP for each class.

C. Results

To assess the effectiveness of integrating YOLO models with the Flower FL framework, we evaluated model performance using mAP. Table IV presents the mAP 0.5 results. The results in Table IV demonstrate that our approach outperforms prior studies integrating YOLO with federated learning. The Flower framework’s combined with YOLO’s advanced model gave us better results. Our non-IID data distribution setup reflects real-world autonomous driving scenarios, enhancing generalization. Dynamic client selection further prioritizes data, model performance. Table IV shows that on the KITTI dataset, our YOLOv8 model combined with FedAvg achieved an impressive mAP of 87.5%, while FedProx achieved a mAP of 87.9%. Both results outperform previous methods, including VINO_EffFedAV (84.3%) and FedPylot (71.8%). Our YOLOv11 model with FedProx reached 84.1%, similar to VINO’s performance. On the BDD100K dataset, YOLOv8 with both FedAvg and FedProx achieved 61.5%, slightly ahead of FedDSL (61.3%) and significantly better than COALA (34.4%).

Federated learning, surpasses centralized training by reducing data transmission costs, optimizing computational efficiency, and ensuring privacy. Federated Learning requires longer training times due to multiple communication rounds. Dataset complexity such as varying lighting, weather conditions, and busy urban scenes with more objects per image impacts performance. While KITTI’s structured scenes lead to higher mAP, the increased complexity of BDD100K results in lower accuracy compared to KITTI. YOLOv8 consistently achieves superior results, on KITTI (87.9%) and BDD (61.5%), compared with YOLOv5 (KITTI: 84.0%, BDD: 60.5%) and YOLOv11 (KITTI: 84.1%, BDD: 60.7%). Our virtual client setup closely simulates real-world autonomous vehicles, validating the effectiveness of integrating Flower with YOLO for decentralized object detection in autonomous driving.

1) *Average Precision across classes:* Class wise analysis of YOLOv8 on KITTI and BDD100K are shown in Table V and VI. As YOLOv8 demonstrates superior performance, examining class-wise results from the YOLOv8 provides deeper insights. This evaluation highlights strengths in detecting each class and identifies challenges with dynamic and underrepresented classes.

TABLE V: KITTI dataset Class-wise AP Scores

Class	mAP-0.5	Precision	Recall
Car	94.6%	88.5%	91%
Van	93.1%	87.2%	86.9%
Truck	97.4%	94.8%	94.6%
Pedestrian	78.8%	92.9%	58.9%
Person Sitting	73.2%	85.9%	58.7%
Cyclist	86.7%	86.8%	79.6%
Tram	91.6%	81.9%	91%
Misc	88%	86.9%	81.2%
Overall	87.9%	88.1%	80.2%

TABLE VI: BDD100K dataset Class-wise AP Scores

Class	mAP-0.5	Precision	Recall
Pedestrian	66.6%	74.6%	60.4%
Rider	50.8%	65.9%	46.1%
Car	79.9%	80.6%	74.3%
Truck	65.4%	66.2%	61.5%
Bus	64.9%	69.1%	58.4%
Motor	49.1%	67.8%	44.6%
Bike	50.9%	60.2%	51.4%
Traffic Light	60.1%	73.3%	53.1%
Traffic Sign	66.4%	70.9%	63.1%
Overall	61.5%	72.9%	51.3%

Well-defined objects with distinct features and larger sizes achieve high detection rates, with Truck (97.4%), Car (94.6%), and Van (93.1%) performing the best. With fewer instances, Tram (91.6%) achieves strong detection with YOLOv8, highlighting that feature uniqueness is a significant factor over object quantity. Detecting smaller

objects remains challenging, as reflected in the lower AP scores for Pedestrians (78.8%) and Person Sitting (73.2%).

The BDD100K dataset, with an overall mAP of 61.5%, includes different lighting, weather conditions, and busy urban scenes that include more objects per image, leading to a decrease in detection performance. Cars (79.9%) remain the most accurately detected class, their AP scores drop significantly compared to KITTI. Trucks (65.4%) and Pedestrians (66.6%) exhibit lower detection performance, largely due to variations in weather and lighting conditions. Small, fast-moving objects, such as Riders (50.8%), Motorcycles (49.1%), and Bicycles (50.9%), have the lowest AP scores, underscoring the challenges posed by these factors.

2) **Inference:** Inference time and frames per second (FPS) are important for understanding model performance, which affects real-time deployment. Table VII presents the results. For KITTI, inference time increased from 0.9s to 1.1s and 1.2s, FPS decreased from 1111 to 909 and 833 across YOLOv5, v8, and 11, respectively. For BDD100K, where inference time increased from 1.3s to 1.6s and 1.7s, while FPS from 769 to 625 and 588 across YOLOv5, v8, and 11, respectively. KITTI (376p) performs faster than BDD100K (720p) due to higher image resolution, increases processing time and poses challenges despite model optimizations.

TABLE VII: Inference Comparison of YOLO versions

Dataset	Inference			FPS		
	v5	v8	v11	v5	v8	v11
KITTI	0.9	1.1	1.2	1111	909	833
BDD100K	1.3	1.6	1.7	769	625	588

V. CONCLUSION

This paper highlights the effectiveness of integrating Federated Learning with YOLO-based object detection using the Flower framework for CAVs. It facilitates decentralized training while preserving data privacy and minimizing communication overhead. The performance evaluation demonstrates that FL-based models achieve competitive performance compared to centralized approaches, making them well-suited for real-world CAV applications. Extending our current study, an evaluation of energy-accuracy trade-offs and resource efficiency across YOLO and Deformable DETR models in federated settings will be presented in our future work.

VI. ACKNOWLEDGEMENT

This work was supported by the U.S. National Science Foundation under award number 2505686 and 2240513.

REFERENCES

- [1] J. Zhao, W. Zhao, B. Deng, Z. Wang, F. Zhang, W. Zheng, W. Cao, J. Nan, Y. Lian, and A. F. Burke, "Autonomous driving system: A comprehensive survey," *Expert Systems with Applications*, vol. 242, p. 122836, 2024.
- [2] D. Jallepalli, N. C. Ravikumar, P. V. Badarinarath, S. Uchil, and M. A. Suresh, "Federated learning for object detection in autonomous vehicles," in *2021 IEEE Seventh International Conference on Big Data Computing Service and Applications (BigDataService)*. IEEE, 2021, pp. 107–114.
- [3] A. M. Elbir, B. Soner, S. Çöleri, D. Gündüz, and M. Bennis, "Federated learning in vehicular networks," in *2022 IEEE International Mediterranean Conference on Communications and Networking (Med-ItCom)*. IEEE, 2022, pp. 72–77.
- [4] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Federated learning for internet of things: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1622–1658, 2021.
- [5] V. P. Chellapandi, L. Yuan, C. G. Brinton, S. H. Žak, and Z. Wang, "Federated learning for connected and automated vehicles: A survey of existing approaches and challenges," *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 119–137, 2023.
- [6] K. Sha, K. S. Cherukuri, and H. Chen, "Privacy and bias in edge computing," in *Edge Computing: Systems and Applications*, L. Xu and W. Shi, Eds. Wiley, 2025, pp. 167–185.
- [7] Z. Du, C. Wu, T. Yoshinaga, K.-L. A. Yau, Y. Ji, and J. Li, "Federated learning for vehicular internet of things: Recent advances and open issues," *IEEE Open Journal of the Computer Society*, vol. 1, pp. 45–61, 2020.
- [8] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão *et al.*, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.
- [9] P. K. Saha, D. Arya, and Y. Sekimoto, "Federated learning-based global road damage detection," *Computer-Aided Civil and Infrastructure Engineering*, vol. 39, no. 14, pp. 2223–2238, 2024.
- [10] M. J. Baucas, P. Spachos, and S. Gregori, "Federated learning platform for secure object recognition in connected and autonomous vehicles," pp. 2306–2311, 2024.
- [11] Y. Li, X. Tao, X. Zhang, J. Liu, and J. Xu, "Privacy-preserved federated learning for autonomous driving," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 7, pp. 8423–8434, 2021.
- [12] K. Vinoth and P. Sasikumar, "Vino_effiedav: Vino with efficient federated learning through selective client updates for real-time autonomous vehicle object detection," *Results in Engineering*, vol. 25, p. 103700, 2025.
- [13] W. Zhuang, J. Xu, C. Chen, J. Li, and L. Lyu, "Coala: A practical and vision-centric federated learning platform," *arXiv preprint arXiv:2407.16560*, 2024.
- [14] H. Zhang, J. Bosch, and H. H. Olsson, "End-to-end federated learning for autonomous driving vehicles," in *2021 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2021, pp. 1–8.
- [15] F. Chi, Y. Wang, P. Nasiopoulos, V. C. Leung, and M. T. Pourazad, "Federated semi-supervised learning for object detection in autonomous driving," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [16] M. Manimaran and V. Dhilipkumar, "Federated diverse self-ensembling learning approach for data heterogeneity in drive vision," *Scalable Computing: Practice and Experience*, vol. 25, no. 6, pp. 4576–4588, 2024.
- [17] A. Khalil, T. Dege, P. Golchin, R. Olshevskiy, A. F. Anta, and T. Meuser, "Federated learning with heterogeneous data handling for robust vehicular object detection," *arXiv preprint arXiv:2405.01108*, 2024.
- [18] E. F. d. S. Soares and C. A. V. Campos, "Privacy-preserving training of monocular depth estimators via self-supervised federated learning," in *2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall)*. IEEE, 2024, pp. 1–6.
- [19] Y. Qian, L. Rao, C. Ma, K. Wei, M. Ding, and L. Shi, "Toward efficient and secure object detection with sparse federated training over internet of vehicles," *IEEE Transactions on Intelligent Transportation Systems*, 2024.
- [20] C. Quéméneur and S. Cherkaoui, "Fedpylot: Navigating federated learning for real-time object detection in internet of vehicles," *arXiv preprint arXiv:2406.03611*, 2024.
- [21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 779–788.
- [22] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The kitti dataset," *The international journal of robotics research*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [23] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "Bdd100k: A diverse driving dataset for heterogeneous multitask learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2636–2645.